

Expected goals from scratch

Logistic regression built by hand, and what log loss does not measure.

7,509 shots · 314 matches · six international tournaments

TRAINING

In: X , 4,435 shots $\times$ 35 features. y in $\{0,1\}$, 1 = scored.

Out: w , 35 weights plus a bias. Plus the training mean and sd of every feature.

DEPLOYMENT

In: one shot's features, scaled with the *stored* training statistics.

Out: one number in $(0, 1)$. No threshold, no label. **The probability is the product.**

THE QUESTION

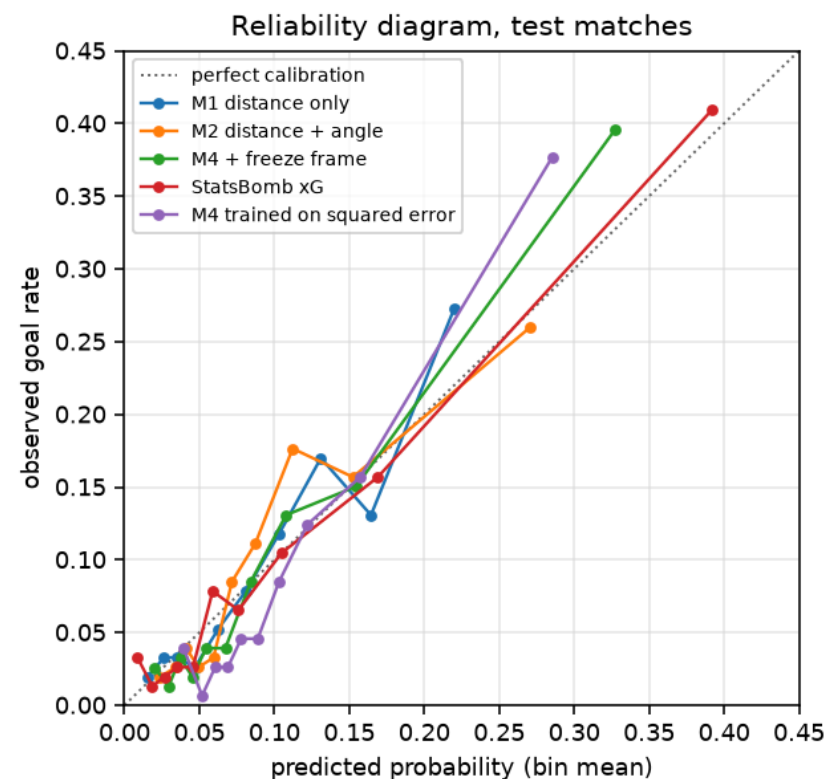
It is trained on one thing and judged on another

An xG model cannot know whether a shot goes in. Two identical chances end differently and neither outcome is a mistake.

What it is actually for is **calibration**: of the chances it calls 0.30, about 30% should be scored.

But it is trained by minimising **log loss over individual binary outcomes**. Those are not the same objective.

Where does that gap bite, how do you detect it, and when you add features, are you buying **calibration** or only **sharpness**?



Reliability diagram, 63 test matches. On the line means the stated percentage matched reality

Start from the perceptron, change two of the three parts

H

Every function $\sigma(w \cdot x + b)$

Unchanged from the perceptron. Choosing weights picks one member.

L

Cross entropy + L2, not on the bias

Proper scoring rule. Gradient survives when the model is confidently wrong.

A

Full-batch gradient descent

$\nabla L = (1/n) X^T (\sigma(Xw) - y) + 2\lambda w$, with the bias row of that penalty set to zero · converged in 501 steps at $\eta = 0.5$

Why not the perceptron. It returns a hard label and I need a probability, so the output type is wrong. And shot data is not separable: the same chance is scored one week and saved the next, so it never converges.

The gradient is error times input. The sigmoid's derivative and the log's derivative cancel, which is why cross entropy and the sigmoid are paired.

`verified` analytic gradient vs central differences: max relative error $4.55e-10$

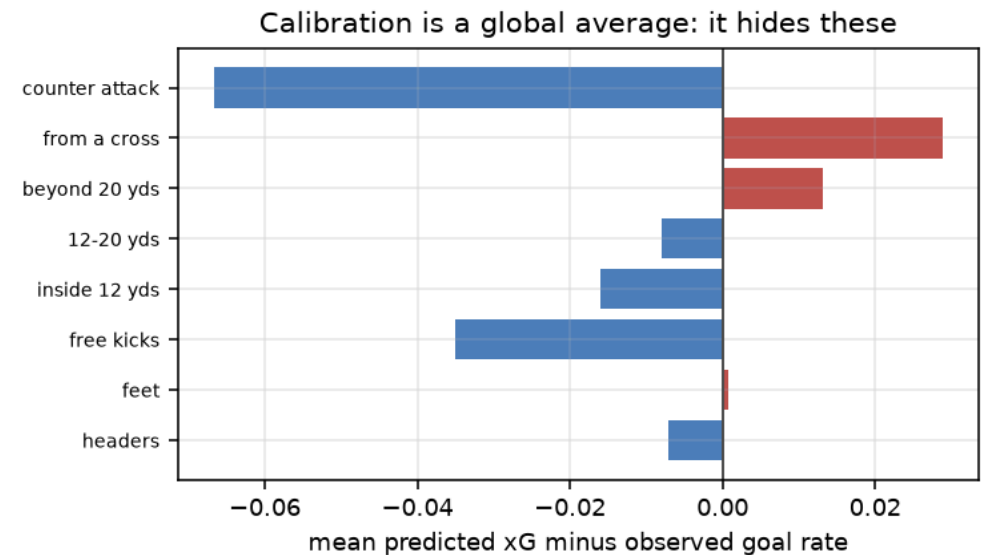
THE FINDING

Perfect on average. Wrong underneath.

SITUATION	SHOTS	MODEL	REALLY	GAP
All shots	1,532	9.29%	9.33%	-0.0
Counter attacks	69	15.1%	21.7%	-6.7
Free kicks	64	2.8%	6.3%	-3.5
From a cross	252	15.2%	12.3%	+2.9
Beyond 20 yds	687	4.2%	2.9%	+1.3

Log loss is a sum over rows. It has **no concept of "counter attack"**, so an underestimate here and an overestimate there are worth the same to it as getting both right.

You cannot find this in the loss. You find it by grouping residuals along dimensions the model never saw.



Mean predicted xG minus observed rate. Overall bias is -0.0004.
None of these are.

Three numbers that carry it

MODEL	FEATURES	LOG LOSS	ECE	ACCURACY
M0 base rate	0	0.3103	0.0032	90.7%
M1 distance	1	0.2792	0.0187	90.7%
M2 + angle	2	0.2774	0.0181	90.7%
M3 + situation	30	0.2605	0.0156	91.0%
M4 + freeze frame	35	0.2584	0.0193	90.8%
StatsBomb commercial	n/a	0.2481	0.0125	91.2%
M4 on squared error	35	0.2668	0.0311	91.0%

1. Sixty lines of NumPy land within **0.011** log loss of a commercial model trained on millions of shots.

2. Predicting "no goal" every time scores **90.7%** accuracy. M3 beats M4 on it. Accuracy is not measuring the task.

3. Changing only the criterion to squared error made calibration **61% worse** while barely moving the ranking. The $p(1-p)$ factor survives and the update stalls where the model is most wrong.

Next: put the grouping into the criterion. A squared group-bias penalty, so the loss contains what the evaluation measures.